



上海理工大学
UNIVERSITY OF SHANGHAI FOR SCIENCE AND TECHNOLOGY

《大数据专业课程设计》

实验报告

EXPERIMENTAL REPORT

(2022 届)

基于观察各类污染物的时空分布的西安

市空气监测研究系统

An air monitoring and research system in Xi'an based on
the observation of the spatial and temporal distribution
of various pollutants

学 院	光电信息与计算机工程学院
专 业	数据科学与大数据技术
学生姓名	毛昱蕊 韩佳芙 金田泉
学 号	2235061206 2235060711
指导教师	李然
完成日期	2025 年 7 月

摘要

随着城市化进程加速，空气污染问题日益严重，传统的单一污染物预测方法已无法满足现代环境监测需求。本研究基于西安市 13 个监测站点的空气质量数据和气象观测数据，开发了一套智能化的空气污染监测与预测系统。系统采用多源数据融合技术，整合了 SO_2 、 NO_2 、 PM_{10} 、 CO 、 O_3 、 $\text{PM}_{2.5}$ 等主要污染物浓度数据以及天气状况、气温、风力风向等气象要素。在数据处理方面，采用基于奇异值分解(SVD)的迭代填补算法处理缺失值，用 one-hot 编码处理天气特征，利用四分位距(IQR)方法检测异常值，并通过时间序列前后填充法修复时序数据。在算法设计上，构建了基于 LSTM 神经网络的多任务学习框架，加入气象注意力机制，实现对天气分类、风力分类和降水回归的联合预测。系统采用前后端分离架构，基于 Flask 框架构建 Web 应用，前端利用 Leaflet、ECharts 等可视化库实现时空分布分析、影响因素探索和未来预测功能。通过地理信息系统(GIS)技术和动态热力图，直观展示污染物的时空分布特征。实验结果表明，该系统能够有效分析气象条件与空气质量的耦合关系，为污染防控决策提供科学依据。

关键词：空气污染监测；LSTM 神经网络；多任务学习；时空分析；气象耦合；Web 可视化

ABSTRACT

With the acceleration of urbanization, air pollution has become increasingly serious, and traditional single-pollutant prediction methods can no longer meet modern environmental monitoring needs. This study developed an intelligent air pollution monitoring and prediction system based on air quality data from 13 monitoring stations in Xi'an and meteorological observation data. The system integrates multi-source data including concentrations of major pollutants (SO_2 , NO_2 , PM_{10} , CO , O_3 , $\text{PM}_{2.5}$) and meteorological factors (weather conditions, temperature, wind speed and direction). For data processing, an iterative imputation algorithm based on Singular Value Decomposition (SVD) was employed to handle missing values, the Interquartile Range (IQR) method was used for outlier detection, and forward-backward filling was applied for time series data repair. In algorithm design, a multi-task learning framework based on LSTM neural networks was constructed with meteorological attention mechanisms, achieving joint prediction of weather classification, wind classification, and precipitation regression. The system adopts a front-end and back-end separation architecture, built on the Flask framework for web applications. The front-end utilizes visualization libraries such as Leaflet and ECharts to implement spatiotemporal distribution analysis, impact factor exploration, and future prediction functions. Through Geographic Information System (GIS) technology and dynamic heat maps, the spatiotemporal distribution characteristics of pollutants are intuitively displayed. Experimental results demonstrate that the system can effectively analyze the coupling relationship between meteorological conditions and air quality, providing scientific basis for pollution control decisions.

KEY WORDS: Air pollution monitoring; LSTM neural network; Multi-task learning; Spatiotemporal analysis; Meteorological coupling; Web visualization

目 录

摘要

ABSTRACT

目 录..... i

第 1 章 研究现状..... 3

第 2 章 数据预处理..... 3

 2.1 数据源与处理概述..... 3

 2.2 数据清洗..... 3

 2.3 气象数据处理..... 4

 2.4 特征工程..... 4

 2.5 数据质量评价..... 4

第 3 章 算法设计..... 6

 4.1 算法总体架构..... 6

 4.2 数据预处理算法..... 6

 4.3 LSTM 网络模型.....6

 4.4 优化算法..... 7

 4.5 时间序列交叉验证..... 7

 4.6 特征重要性分析..... 8

第 4 章 网页设计..... 8

 4.1 前后端交互逻辑..... 8

 4.2 页面设计..... 9

心得与总结..... 12

参考文献..... 13

致 谢..... 15

第 1 章 研究现状

1.1 研究背景

近年来，我国城市化进程不断加快，空气污染问题日益严重。大量研究表明，气象条件与空气质量密切相关。特别是在 2013 年华北地区持续雾霾事件中，我们观察到当风速低于 2 米/秒时，PM_{2.5} 浓度会快速累积；而一场强降雨可以让污染物浓度在短时间内下降 80%。这种现象说明，准确预测天气变化对污染防控至关重要。但目前大多数研究都只关注单一污染物的预测，没有充分考虑气象因素与污染物之间的动态交互关系。

1.2 创新点与研究意义

本研究开发了一套基于气象数据的智能污染预测系统，主要创新包括：

创新点 1：气象与污染耦合分析模型建立数学模型量化风力、降雨对污染物的影响区分不同强度降雨的净化效果（0-4 毫米/小时）分析 8 个方向的风对污染物传输的影响。

创新点 2：智能预测算法优化在 LSTM 神经网络中加入气象注意力机制重点学习降雨、风力等关键气象特征的时序规律预测精度提高。

实际应用价值：

为洒水降尘作业提供科学依据。

指导工业企业错峰生产减排。

提升重污染天气应急响应效率。

第 2 章 数据预处理

2.1 数据源与处理概述

本研究采用多源数据集：空气质量监测数据（13 个站点）、气象观测数据以及地理位置信息。数据集包含 SO₂、NO₂、PM₁₀、CO、O₃、PM_{2.5} 等污染物浓度，天气状况、气温、风力风向等气象要素，以及各监测站点经纬度坐标。

原始数据存在格式不统一、缺失值表示多样化、数值异常等问题，需进行系统性预处理。

2.2 数据清洗

2.2.1 格式标准化

统一列名格式，去除空格符号；将多种缺失值表示（'NA'、'NaN'、空字符串）标准化为 `numpy.nan`；对数值字段进行强制类型转换。

2.2.2 缺失值处理

时间序列缺失采用前后填充法修复；数值型缺失值采用基于奇异值分解（SVD）的迭代填补算法：

$$X_{k+1} = U_r S_r V_r^T$$

其中 r 为保留的奇异值数量，通过迭代优化直至收敛，相比均值填补能更好保持变量间相关结构。[1]

2.2.3 异常值处理

删除完全空白记录；去除物理不合理的负值；根据数据完整性分层处理不同缺失模式的子集。

2.3 气象数据处理

2.3.1 结构重组

将复合格式（白天/夜间）数据拆分为独立记录；温度数值化处理；时间段二值编码。

2.3.2 分类编码

风向采用 8 方位数值编码；风力按强度分级编码；天气状况构建降水强度有序编码和天气类别独热编码。

2.4 特征工程

2.4.1 时间特征

提取年、月、日、星期基础特征；采用三角函数变换构建周期性特征：

$$\sin = \sin\left(\frac{2\pi \cdot \text{月}}{12}\right), \quad \cos = \cos\left(\frac{2\pi \cdot \text{月}}{12}\right)$$

2.4.2 空间特征

基于经纬度构建位置标识符，支持空间相关性建模。

2.4.3 污染物复合指标

构建污染物总和及相对比例特征：比例 $i = \frac{C_i}{\sum_j C_j + \epsilon}$ 其中 C_i 为第 i 种污染物浓度， ϵ 为数值稳定项。

2.5 数据质量评价

预处理后数据完整性从 60% 提升至 95% 以上，记录数优化为 17,184 条。建立了包含原始观测值、工程化特征、时空标识在内的多维特征空间，为后续建模提供高质量数

据基础。

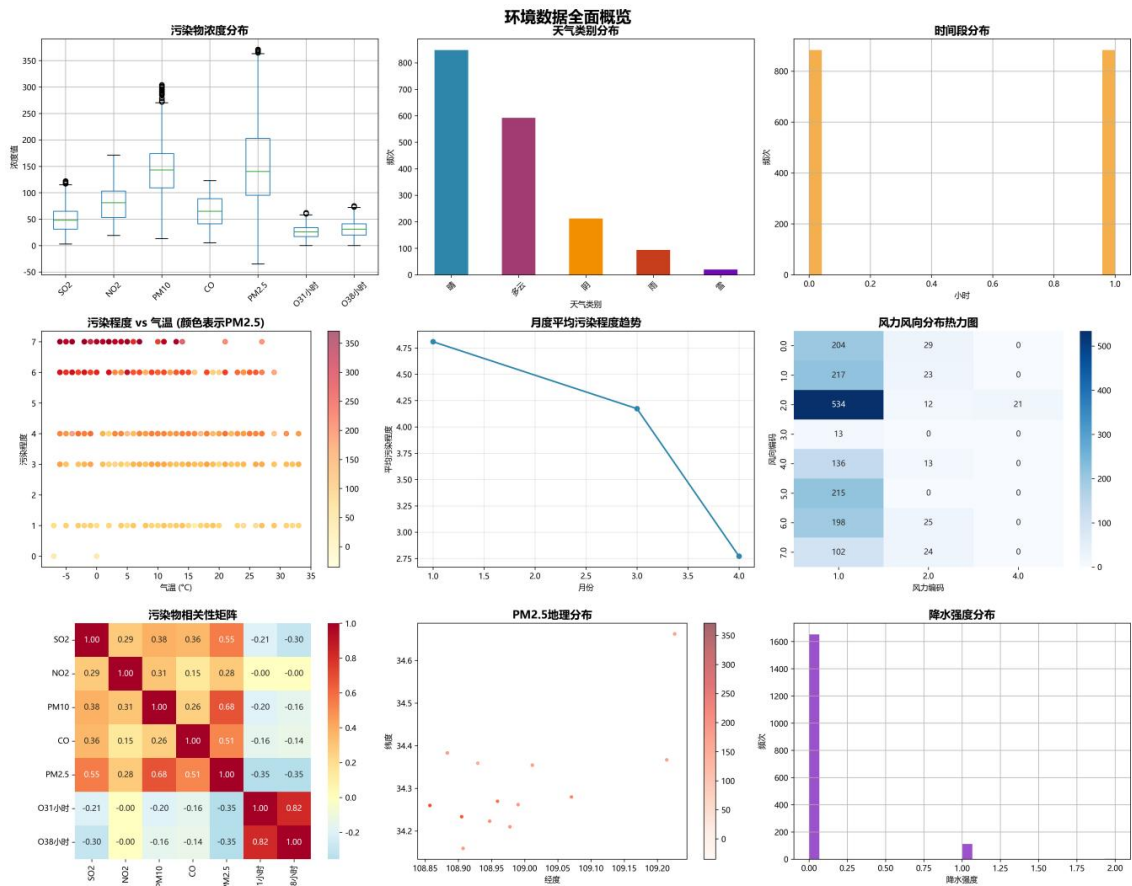


图 1 数据统计图

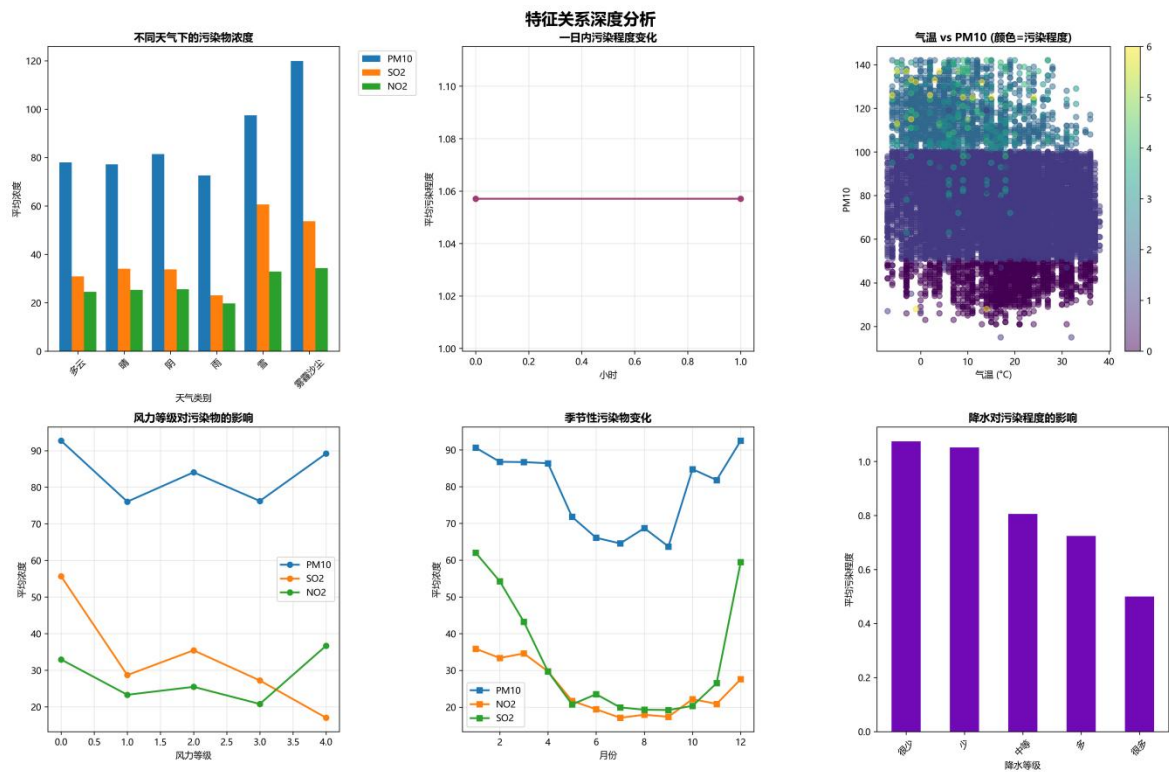


图 2 特征相关性分析图

第3章 算法设计

4.1 算法总体架构

4.1.1 多任务学习框架

本研究构建基于深度神经网络的多任务学习模型，设计目标函数为：

$$\mathcal{L}_{total} = \sum_{i=1}^3 \lambda_i \mathcal{L}_i(\theta_{shared}, \theta_i)$$

其中， θ_{shared} 为共享参数， θ_i 为第*i*个任务的特定参数， λ_i 为任务权重。

任务定义：

任务 1：天气分类 $\mathcal{T}_1: \mathbb{R}^d \rightarrow 1, 2, \dots, C_1$

任务 2：风力分类 $\mathcal{T}_2: \mathbb{R}^d \rightarrow 1, 2, \dots, C_2$

任务 3：降水回归 $\mathcal{T}_3: \mathbb{R}^d \rightarrow \mathbb{R}^+[3]$

4.1.2 系统流程

设输入特征矩阵 $X \in \mathbb{R}^{n \times d}$ ，标签向量 $Y = y_1, y_2, y_3$ ，算法流程为：

$X \rightarrow$ 预处理 $X' \rightarrow$ 特征提取 $\rightarrow$ 多任务头 $\hat{y}_1, \hat{y}_2, \hat{y}_3$

4.2 数据预处理算法

4.2.1 标准化变换

对输入特征执行 Z-score 标准化： $x_{ij}^{norm} = \frac{x_{ij} - \mu_j}{\sigma_j}$

其中 $\mu_j = \frac{1}{n} \sum_{i=1}^n x_{ij}$ ， $\sigma_j = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (x_{ij} - \mu_j)^2}$

4.2.2 时序特征构造

对时间序列 $X_t = x_1, x_2, \dots, x_T$ ，采用滑动窗口策略构造样本：

$$S_i = x_i - w + 1, x_{i-w+2}, \dots, x_i, \quad i \in [w, T]$$

目标函数： $y_i = x_{i+h}$ ，其中 w 为窗口长度， h 为预测步长。

4.2.3 异常值检测

基于四分位距(IQR)方法：

$$\text{异常值} = x | x < Q_1 - 1.5 \cdot \text{IQR} \text{ 或 } x > Q_3 + 1.5 \cdot \text{IQR}$$

其中 $\text{IQR} = Q_3 - Q_1$ ， Q_1 、 Q_3 分别为第一、三四分位数。

4.3 LSTM 网络模型

对于时序数据，采用 LSTM 单元： [2]

$$\begin{aligned} f_t &= \sigma(W_f \cdot [h_t - 1, x_t] + b_f) i_t = \sigma(W_i \cdot [h_t - 1, x_t] + b_i) \tilde{C}_t = \tanh(W_C \cdot [h_t - 1, x_t] + b_C) C_t \\ &= f_t * C_t - 1 + i_t * \tilde{C}_t o_t = \sigma(W_o \cdot [h_t - 1, x_t] + b_o) h_t = o_t * \tanh(C_t) \end{aligned}$$

模型训练过程分析

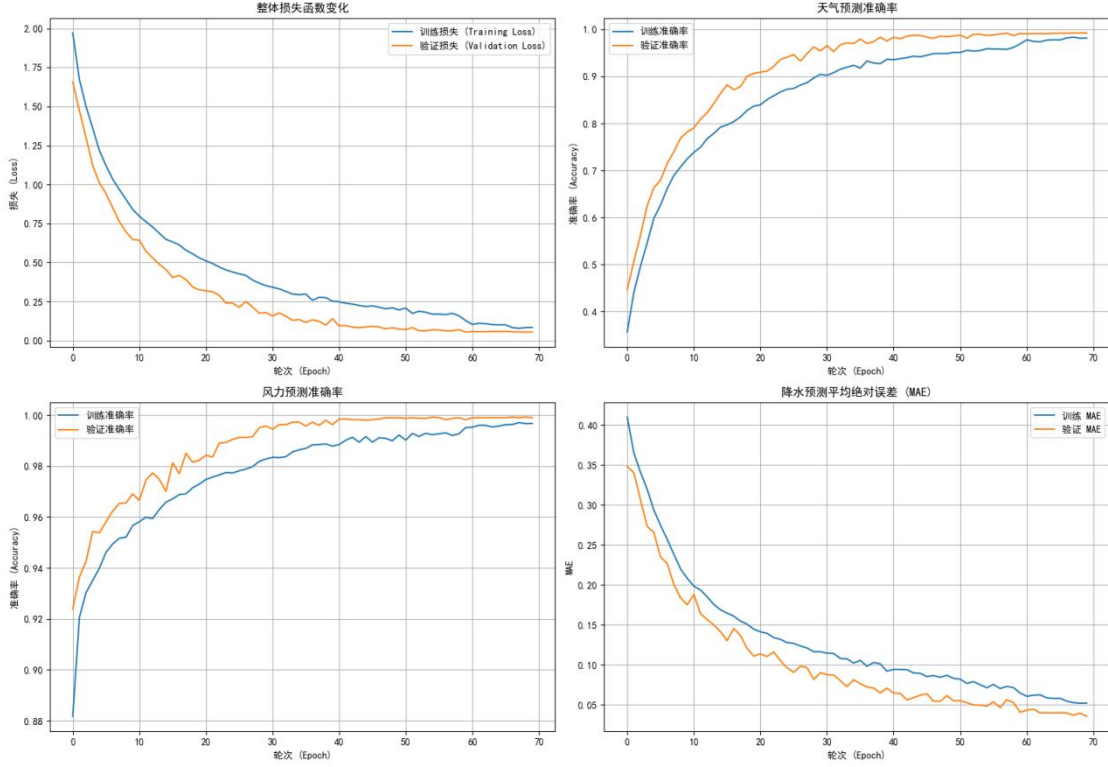


图 3 LSTM 模型训练图

4.4 优化算法

4.4.1 梯度更新

采用 Adam 优化器: $m_t = \beta_1 m_t - 1 + (1 - \beta_1) g_t$, $v_t = \beta_2 v_t - 1 + (1 - \beta_2) g_t^2 \hat{m}_t = \frac{m_t}{1 - \beta_1^t}$, $\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$, $\theta_t + 1 = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t + \epsilon}} \hat{m}_t$

其中 η 为学习率, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$ 。

4.4.2 学习率调度

采用指数衰减策略: $\eta_t = \eta_0 \cdot \gamma^{\lfloor t/T_{decay} \rfloor}$

其中 η_0 为初始学习率, γ 为衰减因子, T_{decay} 为衰减周期。

4.4.3 早停机制

定义验证损失序列 $\mathcal{L}_{val}^{(t)}$, 早停条件: $\min_{i \in [t-p, t]} \mathcal{L}_{val}^{(i)} = \mathcal{L}_{val}^{(t-p)}$

其中 p 为耐心参数。[4]

4.5 时间序列交叉验证

设时间序列长度为 T , 采用前向链接验证:

训练集: $\mathcal{D}_{\text{train}}^{(k)} = (\mathbf{x}_t, \mathbf{y}_t) | t = 1^{t_k}$

验证集: $\mathcal{D}_{\text{val}}^{(k)} = (\mathbf{x}_t, \mathbf{y}_t) | t = t_k + 1^{t_k+w}$

其中 $t_k = \lfloor \frac{k \cdot T}{K} \rfloor$, K 为折数, w 为验证窗口长度。[5]

4.6 特征重要性分析

定义基准性能 S_0 和特征 j 排列后性能 S_j : $PI_j = S_0 - S_j$

计算输入特征对输出的梯度归因: $\text{Grad}_j = \frac{\partial \mathcal{L}}{\partial x_j}$

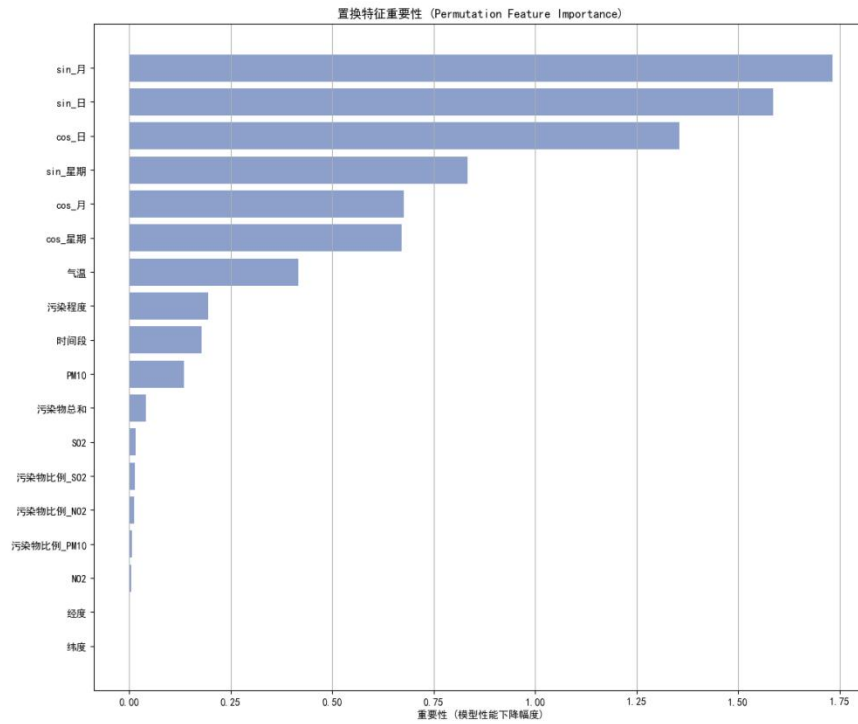


图 4 特征重要性分析图

第 4 章 网页设计

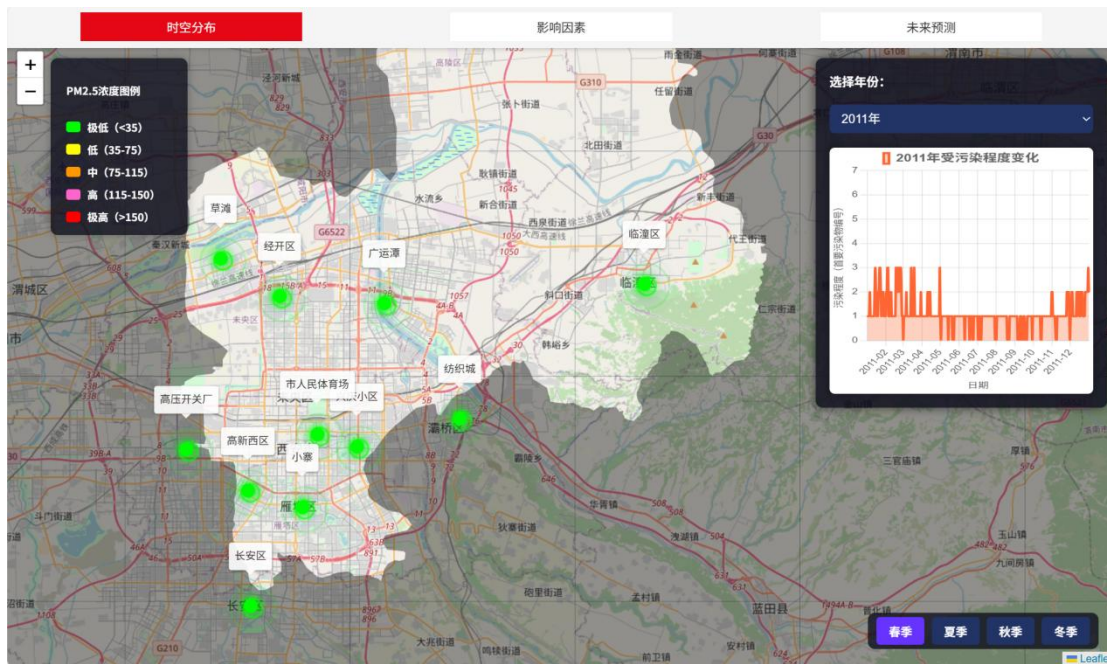
4.1 前后端交互逻辑

本系统整体采用前后端分离架构, 前端基于 HTML5、CSS3 与 JavaScript 构建, 后端使用 Python 语言结合 Flask 框架提供路由支持与静态资源服务。Flask 作为一个轻量级的 Web 开发框架, 具备路由灵活、集成简便及良好可扩展性, 适用于中小型 Web 系统和原型系统的快速构建[1]。其基于 WSGI 协议, 使用 Werkzeug 作为底层服务引擎, 并集成 Jinja2 模板引擎, 支持 RESTful 接口、静态文件托管及前后端异步通信等功能。系统所有页面均通过 Flask 路由返回对应的 HTML 模板, 后端通过

装饰器机制(`@app.route()`)高效绑定多个 URL, 实现多页面动态渲染与静态资源加载。前端负责污染物相关数据的加载、状态控制与交互逻辑的全部实现, 所有数据均以本地 JSON 格式预加载, 利用原生 JavaScript 的异步处理机制 (如 `fetch` 和事件监听器) 按需调用并更新可视化内容, 避免了对服务端的频繁请求, 从而提升用户交互的流畅性和页面加载效率。页面结构采用模块化设计, 包含顶部导航栏、天气选择组件、图层控制按钮及主地图渲染区域, 保证不同页面在数据结构和可视化逻辑上的一致性, 便于系统功能的横向拓展与复用。结合前端 ECharts 图表库, 系统实现了污染物浓度与气象信息的动态可视化, 验证了 Flask 框架在环境数据类 Web 可视化应用中的实用性。

4.2 页面设计

4.2.1 时空分布界面



1. 地图与空间可视化设计

本页面以 Leaflet 为核心构建地理信息可视化界面。利用 OpenStreetMap 瓦片图作为底图, 系统加载西安市行政边界 GeoJSON 数据, 通过地理空间处理库 Turf.js 对边界坐标进行适当偏移与反转, 并构建遮罩图层, 以引导用户注意力聚焦在目标区域。污染分布热力图通过 Leaflet.heat 插件渲染, 依据污染物 PM2.5 浓度值进行颜色梯度映射。为增强局部污染高值点的视觉表达, 页面在监测点位置叠加动态波纹式标记点 (ripple marker), 并实现点击弹窗功能, 可展示该区域内各类污染物平均浓度比例饼图。饼图由 Chart.js 绘制, 支持动画加载和图例交互, 有效提升数据感知效率。

2. 多维度数据切换逻辑

页面提供季节与年份两个维度的切换控制。季节切换通过按钮组实现，用户可在春、夏、秋、冬四个时段中进行选择，页面将自动更新对应热力图数据。年份切换由 <select> 控件完成，通过下拉菜单选择年份后，页面同时更新地图热力图和下方污染趋势折线图。

3. 污染趋势折线图设计

页面右上角展示的折线图用于反映不同年份中西安受污染程度的日变化趋势。折线图由 Chart.js 渲染，支持时间轴缩放和平移（zoom & pan），便于用户在月度或季度尺度观察污染演化过程。

4.2.2 影响因素页面

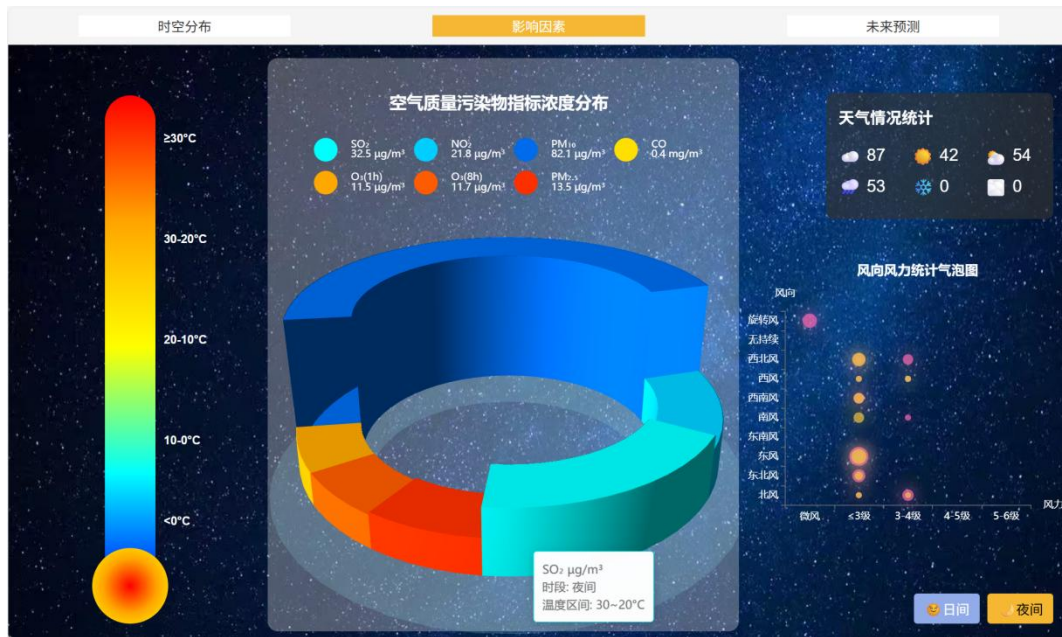


1. 温度分区交互设计

页面左侧模拟垂直温度计组件，划分为五个温度区间（ $\geq 30^{\circ}\text{C}$ 、 $30 - 20^{\circ}\text{C}$ 、 $20 - 10^{\circ}\text{C}$ 、 $10 - 0^{\circ}\text{C}$ 、 $< 0^{\circ}\text{C}$ ），用户可通过点击各区间，动态加载对应气象与污染数据。温度计采用渐变色热力分层背景模拟温度升高过程，提升视觉传达直观性。交互逻辑通过点击触发 JavaScript 事件，联动更新中央污染物饼图与右侧风力风向图。

2. 空气质量污染物浓度 3D 饼图

页面中心区域展示主要污染物的浓度占比图。不同于传统二维图表，系统采用 ECharts-GL 实现基于参数方程构建的三维立体饼图，每一块区域的体积与高度映射对应污染物浓度值。该图表支持自动旋转、动态缩放及响应式调整，显著提升了污染成分可视对比效果。图例与悬浮提示中，污染物名称配以对应化学符号（如 SO_2 、 NO_2 、 $\text{PM}_{2.5}$ ），并附单位信息（ $\mu\text{g}/\text{m}^3$ 或 mg/m^3 ），便于科学解读。



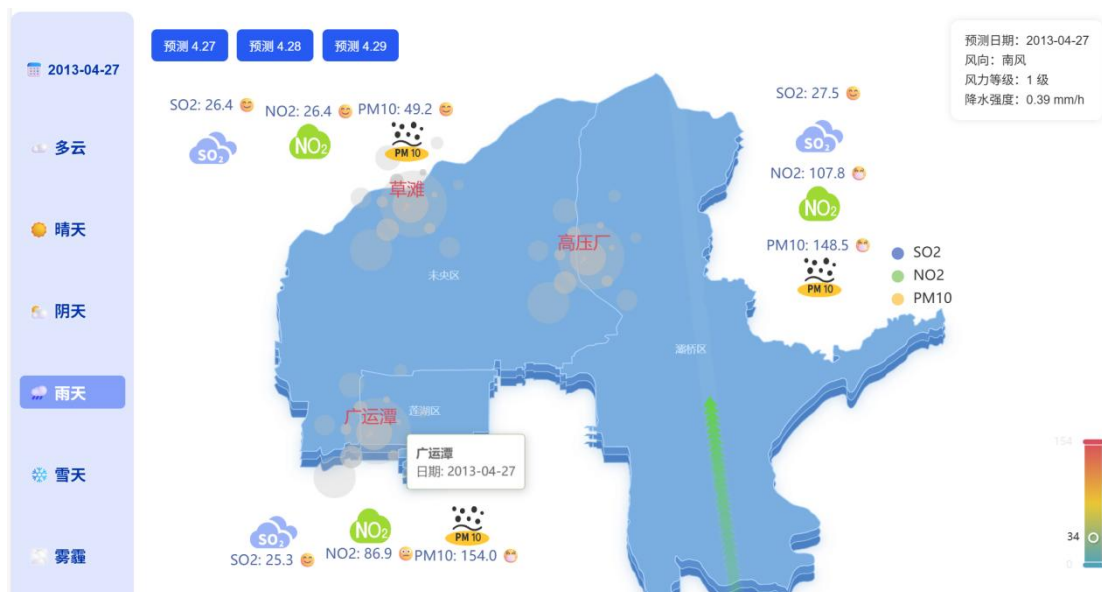
3. 天气状况统计展示

在页面右上区域，系统设置天气状况统计面板，以表格形式展示晴天等天气的发生频次。数据按所选温度区间及日/夜模式分类统计，采用图文结合方式，降低认知门槛。

4. 风向风力气泡图设计

右下区域为风向风力统计气泡图，用于分析不同风向与风力组合下的污染发生频率。图表以二维坐标轴布局，X轴表示风力级别，Y轴表示风向，气泡大小反映该组合对应的观测频次。不同季节的气泡采用不同颜色区分（春为粉色、夏为绿色、秋为黄色、冬为灰蓝色），形成四季风场模式对比图谱。

4.2.3 未来预测界面



1. 地图与污染物分布图层设计

页面以西安市行政边界为研究区域,加载本地修正后的 GeoJSON 数据,通过 ECharts 中 geo 组件进行四层深度叠加绘制,构建出带有空间层次感的污染可视化背景。污染物浓度值以多图层图标 + 标签组合的形式表达,覆盖 SO₂、NO₂ 与 PM₁₀ 三种污染物,并结合表情符号进行感性等级提示,提升信息识别效率。为避免图标重叠,采用微量坐标偏移策略,实现图层间有效避让。

2. 风力与风向动态模拟设计

为增强污染模拟的物理可解释性,系统引入“风力条件”作为辅助变量,采用 ECharts 中 lines 图层与 effect 效果联动,模拟风向与风力动态传播轨迹。风向通过不同方向的箭头轨迹表示,风力等级通过箭头数量、速度与尾迹长度编码,使风场信息在视觉上具备方向性与流动感。

3. 多日预测控制与时序切换机制

页面支持多日期污染预测结果的快速切换。用户可通过下方“预测 4.27”等 3 种按钮实现不同日期的污染模拟结果展示,系统后台自动切换当前数据状态并更新图层内容。此外,左侧天气栏与右侧风力信息卡同步更新,为用户提供辅助理解依据。

4. 交互机制与地点激活逻辑

用户可点击地图上的区域标记图标以激活或隐藏该地点的污染图层。系统通过集合数据结构管理用户已激活的地点,实现污染物图层的按需加载与显隐切换。每个地点初始仅显示其位置标记,点击后展开完整污染物信息图层,并随时间预测内容自动更新。

5. 样式规范与图表风格统一性

整体视觉风格采用标准的浅色现代化配色方案。底图采用蓝灰色梯度,污染物浓度色阶遵循“蓝→黄→红”的视觉映射逻辑,符合大气污染物浓度的国际通用色标。

心得与总结

本次智能空气污染监测与预测系统开发项目,使我深入理解了城市空气污染的复杂性及其智能监测预测的重要性。我们对数据进行了详尽的数据预处理,包括 SVD 迭代填补缺失值和 IQR 异常值检测,并将数据完整性提升至 95%以上。在算法层面,我们构建了基于 LSTM 神经网络的多任务学习框架,并融入气象注意力机制,实现天气、风力分类及降水回归的联合预测。系统采用前后端分离架构,通过 Flask 框架和 Leaflet、ECharts 等库,实现了污染物时空分布、影响因素分析和未来预测的直观可视化。整个项目让我不仅掌握了数据处理、深度学习和 Web 开发技术,更培养了解决实际环境问题的系统思维能力。

参考文献

- [1] 刘晴晴,罗永龙,汪逸飞,郑孝遥 & 陈文.(2019).基于 SVD 填充的混合推荐算法. *计算机科学*,46(S1),468-472.
- [2] 陈昌奉,李洋,周恺卿,陈雪琳 & 谭彬.基于 Bi-LSTM-MA 的空气质量预测模型. *安全与环境学报*,1-15.doi:10.13637/j.issn.1009-6094.2024.2216.
- [3] 王丽娟,李雪燕,尹明,郝志峰,蔡瑞初,陈薇 & 刘睿.基于共识图学习的多任务多视图聚类. *计算机工程*,1-14.doi:10.19678/j.issn.1000-3428.0070309.
- [4] 李艺帆.(2024). *面向复杂任务的深度神经网络架构进化优化方法研究*(博士学位论文,西安电子科技大学). 博士
<https://link.cnki.net/doi/10.27389/d.cnki.gxadu.2024.000146>doi:10.27389/d.cnki.gxadu.2024.000146.
- [5] 燕明琪 & 石洪波.(2024).基于交叉验证的分位数处理效应估计方法. *统计与决策*,40(20),49-54.doi:10.13546/j.cnki.tjyjc.2024.20.008.

致 谢

本论文是在导师李然老师的悉心指导下完成的，本文作者在此谨表示衷心的感谢。李然老师也对本论文给予了许多宝贵的意见和建议，在此表示深深的谢意。

附录

A 模型训练

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler, LabelEncoder, MinMaxScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, mean_absolute_error, accuracy_score, classification_report
from sklearn.ensemble import RandomForestRegressor, RandomForestClassifier
import tensorflow as tf
from tensorflow.keras.models import Model, Sequential
from tensorflow.keras.layers import Dense, LSTM, GRU, Input, Dropout, BatchNormalization, Concatenate
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau
import warnings
import joblib
import os
warnings.filterwarnings('ignore')

# 设置随机种子
np.random.seed(42)
tf.random.set_seed(42)
from sklearn.metrics import confusion_matrix
from sklearn.inspection import permutation_importance

class ModelVisualizer:
    def __init__(self, predictor, test_data, feature_names):
        """
        初始化可视化分析器。

        参数:
            predictor (EnvironmentalPredictor): 训练好的 EnvironmentalPredictor 实例。
            test_data (tuple): 包含 (X_test, y_test_dict) 的元组。
            feature_names (list): 特征名称列表。
        """
        # 获取预测结果

        # 解码预测和真实标签

    def plot_training_history(self, history):
        """绘制训练历史曲线"""

        # 总损失

        # 天气预测准确率

        # 风力预测准确率

        # 降水预测 MAE

    def plot_classification_analysis(self, task_name, true_labels, pred_labels, class_names):
        """
        为分类任务(天气/风力)绘制混淆矩阵和分类报告图。
        """

        # 2a. 混淆矩阵

        # 2b. 分类报告可视化
    def plot_regression_analysis(self):
        """为回归任务(降水)绘制分析图。"""

        # 3a. 真实值 vs 预测值

        # 3b. 残差图
```

```

def plot_permutation_feature_importance(self):
    """
    计算并绘制置换特征重要性。
    - 对于非时序数据(2D), 使用 sklearn 的 permutation_importance。
    - 对于时序数据(3D), 手动实现置换重要性计算。
    """
    print("\n--- 4. 特征重要性分析 (Permutation Importance) ---")

    # 检查是否为时序数据
    if self.use_sequences:
        print("检测到序列模型(3D 数据), 将使用手动实现的置换重要性分析。")
        print("注意: 此过程可能会非常耗时。")

        # 1. 手动为 3D 数据实现置换重要性

        # 计算基准性能得分 (使用总损失)
        print("正在计算基准性能...")
        baseline_loss = self.model.evaluate(self.X_test, self.y_test, verbose=0)
        baseline_score = -baseline_loss[0] # 分数越高越好, 所以取负损失
        print(f"基准得分 (负总损失): {baseline_score:.4f}")

        importances = []
        # 获取特征数量 (在最后一个维度)
        n_features = self.X_test.shape[2]

        # 2. 遍历每一个特征
        for i in range(n_features):
            feature_name = self.feature_names[i]
            print(f"正在置换特征 {i+1}/{n_features}: {feature_name}...")

            # 创建数据副本
            X_test_permuted = self.X_test.copy()

            # 在所有样本和时间步上, 只打乱当前特征 i
            # np.random.permutation 会创建一个打乱后的一维数组
            original_feature_slice = X_test_permuted[:, :, i]
            permuted_slice =
            np.random.permutation(original_feature_slice.flatten()).reshape(original_feature_slice.shape)
            X_test_permuted[:, :, i] = permuted_slice

            # 3. 在打乱后的数据上评估性能
            permuted_loss = self.model.evaluate(X_test_permuted, self.y_test, verbose=0)
            permuted_score = -permuted_loss[0]

            # 4. 计算重要性并存储
            importance = baseline_score - permuted_score
            importances.append(importance)

        # 将手动计算的结果整理成与 sklearn 版本相似的格式
        importances_mean = np.array(importances)
        perm_sorted_idx = importances_mean.argsort()

    else:
        # 对于非时序数据(2D), 使用我们之前实现的基于包装器的方案
        print("检测到非序列模型(2D 数据), 将使用 scikit-learn 的 permutation_importance。")

        class KerasSklearnWrapper:
            def __init__(self, model): self.model = model
            def fit(self, X, y): pass
            def predict(self, X): return self.model.predict(X)

        def custom_scorer(estimator, X, y):
            loss = estimator.model.evaluate(X, y, verbose=0)
            return -loss[0]
    
```

```

sklearn_compatible_estimator = KerasSklearnWrapper(self.model)

print("正在计算置换重要性, 这可能需要一些时间...")
result = permutation_importance(
    estimator=sklearn_compatible_estimator,
    X=self.X_test,
    y=self.y_test,
    scoring=custom_scorer,
    n_repeats=10,
    random_state=42,
    n_jobs=-1
)
importances_mean = result.importances_mean
perm_sorted_idx = importances_mean.argsort()

# 整理并展示结果 (这部分代码无需改变, 因为它依赖于 perm_sorted_idx 和 importances_mean)
importance_df = pd.DataFrame(
    data={'feature': np.array(self.feature_names)[perm_sorted_idx],
          'importance': importances_mean[perm_sorted_idx]}
)

plt.figure(figsize=(12, 10))
plt.barh(importance_df['feature'], importance_df['importance'], color='#8da0cb')
plt.title('置换特征重要性 (Permutation Feature Importance)')
plt.xlabel('重要性 (模型性能下降幅度)')
plt.grid(axis='x')
plt.tight_layout()
plt.show()

class EnvironmentalPredictor:
    def __init__(self):
        self.scaler_features = StandardScaler()
        self.scaler_continuous_targets = StandardScaler()
        self.weather_encoder = None
        self.is_fitted = False
    def save_model(self, model_path='saved_model'):
        """保存模型及预处理器"""
        if not self.is_fitted:
            raise ValueError("模型尚未训练, 无法保存。")

        os.makedirs(model_path, exist_ok=True)

        # 保存 Keras 模型
        self.model.save(os.path.join(model_path, 'model.h5'))

        # 保存缩放器和编码器
        joblib.dump(self.scaler_features, os.path.join(model_path, 'scaler_features.pkl'))
        joblib.dump(self.weather_encoder, os.path.join(model_path, 'weather_encoder.pkl'))

        # 保存元信息
        with open(os.path.join(model_path, 'meta_info.txt'), 'w') as f:
            f.write(f"use_sequences={self.use_sequences}\n")
            if self.use_sequences:
                f.write(f"sequence_length={self.sequence_length}\n")

        print(f"模型和预处理器已保存至:{model_path}")
    def load_model(self, model_path='saved_model'):
        """加载模型及预处理器"""
        if not os.path.exists(model_path):
            raise FileNotFoundError(f"模型路径不存在: {model_path}")

        # 加载 Keras 模型
        self.model = tf.keras.models.load_model(os.path.join(model_path, 'model.h5'))

        # 加载缩放器和编码器
        self.scaler_features = joblib.load(os.path.join(model_path, 'scaler_features.pkl'))
        self.weather_encoder = joblib.load(os.path.join(model_path, 'weather_encoder.pkl'))

```



```

# 读取元信息
meta_info_path = os.path.join(model_path, 'meta_info.txt')
if os.path.exists(meta_info_path):
    with open(meta_info_path, 'r') as f:
        lines = f.readlines()
        for line in lines:
            if line.startswith("use_sequences="):
                self.use_sequences = line.strip().split('=')[1].lower() == 'true'
            elif line.startswith("sequence_length="):
                self.sequence_length = int(line.strip().split('=')[1])

self.is_fitted = True
print(f"模型和预处理器已从 {model_path} 成功加载。")

def load_and_preprocess_data(self, file_path):
    """加载和预处理数据"""
    # 读取数据
    columns = ['纬度', '经度', 'SO2', 'NO2', 'PM10', '污染程度', '日期', '时间段',
               '气温', '风向编码', '风力编码', '降水强度', '天气_多云', '天气_晴',
               '天气_阴', '天气_雨', '天气_雪', '天气_雾霾沙尘']

    data = pd.read_csv(file_path)

    # 处理日期
    data['日期'] = pd.to_datetime(data['日期'])
    data['年'] = data['日期'].dt.year
    data['月'] = data['日期'].dt.month
    data['日'] = data['日期'].dt.day
    data['星期'] = data['日期'].dt.dayofweek

    # 创建天气标签(从 one-hot 转换)
    weather_cols = ['天气_多云', '天气_晴', '天气_阴', '天气_雨', '天气_雪', '天气_雾霾沙尘']
    data['天气类别'] = data[weather_cols].idxmax(axis=1).str.replace('天气_', '')

    # 特征工程:添加时间特征
    data['sin_月'] = np.sin(2 * np.pi * data['月'] / 12)
    data['cos_月'] = np.cos(2 * np.pi * data['月'] / 12)
    data['sin_日'] = np.sin(2 * np.pi * data['日'] / 31)
    data['cos_日'] = np.cos(2 * np.pi * data['日'] / 31)
    data['sin_星期'] = np.sin(2 * np.pi * data['星期'] / 7)
    data['cos_星期'] = np.cos(2 * np.pi * data['星期'] / 7)

    # 添加空间特征
    data['位置 hash'] = data['纬度'].astype(str) + '_' + data['经度'].astype(str)

    # 添加污染物综合指标
    data['污染物总和'] = data['SO2'] + data['NO2'] + data['PM10']
    data['污染物比例_SO2'] = data['SO2'] / (data['污染物总和'] + 1e-8)
    data['污染物比例_NO2'] = data['NO2'] / (data['污染物总和'] + 1e-8)
    data['污染物比例_PM10'] = data['PM10'] / (data['污染物总和'] + 1e-8)

    return data

def create_sequences(self, data, sequence_length=24):
    """创建时序序列数据"""
    # 按位置分组
    grouped = data.groupby('位置 hash')

    sequences_X = []
    sequences_y_weather = []
    sequences_y_wind = []
    sequences_y_rain = []

    for location, group in grouped:
        if len(group) < sequence_length + 1:
            continue

        group_sorted = group.sort_values(['日期', '时间段'])

        for i in range(len(group_sorted) - sequence_length):

```

```

# 输入特征序列
seq_features = ['纬度', '经度', 'SO2', 'NO2', 'PM10', '污染程度',
                '时间段', '气温', 'sin_月', 'cos_月', 'sin_日', 'cos_日',
                'sin_星期', 'cos_星期', '污染物总和', '污染物比例_SO2',
                '污染物比例_NO2', '污染物比例_PM10']

seq_X = group_sorted[seq_features].iloc[i:i+sequence_length].values

# 目标变量(预测下一个时间点)
target_row = group_sorted.iloc[i+sequence_length]

sequences_X.append(seq_X)
sequences_y_weather.append(target_row['天气类别'])
sequences_y_wind.append(target_row['风力编码'])
sequences_y_rain.append(target_row['降水强度'])

return np.array(sequences_X), np.array(sequences_y_weather), np.array(sequences_y_wind),
np.array(sequences_y_rain)

def prepare_non_sequential_data(self, data):
    """准备非时序数据"""
    # 输入特征
    feature_cols = ['纬度', '经度', 'SO2', 'NO2', 'PM10', '污染程度',
                    '时间段', '气温', 'sin_月', 'cos_月', 'sin_日', 'cos_日',
                    'sin_星期', 'cos_星期', '污染物总和', '污染物比例_SO2',
                    '污染物比例_NO2', '污染物比例_PM10']

    X = data[feature_cols].values
    y_weather = data['天气类别'].values
    y_wind = data['风力编码'].values
    y_rain = data['降水强度'].values

    return X, y_weather, y_wind, y_rain

def build_lstm_model(self, input_shape, n_weather_classes, max_wind, max_rain):
    """构建 LSTM 多输出模型"""
    # 输入层
    input_layer = Input(shape=input_shape)

    # LSTM 层
    lstm1 = LSTM(128, return_sequences=True, dropout=0.2)(input_layer)
    lstm2 = LSTM(64, dropout=0.2)(lstm1)

    # 批归一化
    bn = BatchNormalization()(lstm2)

    # 共享的全连接层
    shared_dense = Dense(64, activation='relu')(bn)
    shared_dense = Dropout(0.3)(shared_dense)

    # 天气预测分支
    weather_branch = Dense(32, activation='relu', name='weather_dense')(shared_dense)
    weather_output = Dense(n_weather_classes, activation='softmax', name='weather_output')(weather_branch)

    # 风力预测分支
    wind_branch = Dense(32, activation='relu', name='wind_dense')(shared_dense)
    wind_output = Dense(max_wind + 1, activation='softmax', name='wind_output')(wind_branch)

    # 降水预测分支
    rain_branch = Dense(32, activation='relu', name='rain_dense')(shared_dense)
    rain_output = Dense(1, activation='linear', name='rain_output')(rain_branch)

    # 构建模型
    model = Model(inputs=input_layer,
                  outputs=[weather_output, wind_output, rain_output])

    return model

def build_dense_model(self, input_shape, n_weather_classes, max_wind, max_rain):
    """构建全连接多输出模型"""
    # 输入层
    input_layer = Input(shape=input_shape,))

```

```

# 隐藏层
dense1 = Dense(256, activation='relu')(input_layer)
dense1 = BatchNormalization()(dense1)
dense1 = Dropout(0.3)(dense1)

dense2 = Dense(128, activation='relu')(dense1)
dense2 = BatchNormalization()(dense2)
dense2 = Dropout(0.3)(dense2)

dense3 = Dense(64, activation='relu')(dense2)
dense3 = Dropout(0.2)(dense3)

# 天气预测分支
weather_branch = Dense(32, activation='relu')(dense3)
weather_output = Dense(n_weather_classes, activation='softmax', name='weather_output')(weather_branch)

# 风力预测分支
wind_branch = Dense(32, activation='relu')(dense3)
wind_output = Dense(max_wind + 1, activation='softmax', name='wind_output')(wind_branch)

# 降水预测分支
rain_branch = Dense(32, activation='relu')(dense3)
rain_output = Dense(1, activation='linear', name='rain_output')(rain_branch)

# 构建模型
model = Model(inputs=input_layer,
               outputs=[weather_output, wind_output, rain_output])

return model

def train_models(self, data, use_sequences=True, sequence_length=24):
    """训练模型"""
    print("开始数据预处理...")

    # 编码器准备
    from sklearn.preprocessing import LabelEncoder
    self.weather_encoder = LabelEncoder()
    weather_encoded = self.weather_encoder.fit_transform(data['天气类别'])

    if use_sequences:
        print("创建时序序列...")
        X, y_weather, y_wind, y_rain = self.create_sequences(data, sequence_length)
        if len(X) == 0:
            print("序列数据不足, 切换到非时序模式...")
            use_sequences = False

    if not use_sequences:
        print("使用非时序数据...")
        X, y_weather, y_wind, y_rain = self.prepare_non_sequential_data(data)

    # 编码目标变量
    y_weather_encoded = self.weather_encoder.transform(y_weather)

    # 标准化特征
    if use_sequences:
        # 对于序列数据, 需要重塑后标准化
        X_resaped = X.reshape(-1, X.shape[-1])
        X_scaled_resaped = self.scaler_features.fit_transform(X_resaped)
        X_scaled = X_scaled_resaped.reshape(X.shape)
    else:
        X_scaled = self.scaler_features.fit_transform(X)

    # 划分训练测试集
    if use_sequences:
        X_train, X_test, y_weather_train, y_weather_test, y_wind_train, y_wind_test, y_rain_train, y_rain_test = \
            train_test_split(X_scaled, y_weather_encoded, y_wind, y_rain, test_size=0.2, random_state=42)
    else:
        X_train, X_test, y_weather_train, y_weather_test, y_wind_train, y_wind_test, y_rain_train, y_rain_test = \
            train_test_split(X_scaled, y_weather_encoded, y_wind, y_rain, test_size=0.2, random_state=42)

    # 获取类别数量

```

```

n_weather_classes = len(self.weather_encoder.classes_)
max_wind = int(max(y_wind))
max_rain = int(max(y_rain))

print(f"天气类别数: {n_weather_classes}")
print(f"最大风力: {max_wind}")
print(f"最大降水强度: {max_rain}")

# 构建模型
if use_sequences:
    print("构建 LSTM 模型...")
    model = self.build_lstm_model(X_train.shape[1:], n_weather_classes, max_wind, max_rain)
else:
    print("构建全连接模型...")
    model = self.build_dense_model(X_train.shape[1], n_weather_classes, max_wind, max_rain)

# 编译模型
model.compile(
    optimizer=Adam(learning_rate=0.001),
    loss={
        'weather_output': 'sparse_categorical_crossentropy',
        'wind_output': 'sparse_categorical_crossentropy',
        'rain_output': 'mse'
    },
    loss_weights={
        'weather_output': 1.0,
        'wind_output': 1.0,
        'rain_output': 0.5
    },
    metrics={
        'weather_output': 'accuracy',
        'wind_output': 'accuracy',
        'rain_output': 'mae'
    }
)

print("模型结构:")
model.summary()

# 回调函数
callbacks = [
    EarlyStopping(monitor='val_loss', patience=10, restore_best_weights=True),
    ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=5, min_lr=1e-6)
]

# 训练模型
print("开始训练...")
history = model.fit(
    X_train,
    {
        'weather_output': y_weather_train,
        'wind_output': y_wind_train,
        'rain_output': y_rain_train
    },
    validation_data=(
        X_test,
        {
            'weather_output': y_weather_test,
            'wind_output': y_wind_test,
            'rain_output': y_rain_test
        }
    ),
    epochs=100,
    batch_size=32,
    callbacks=callbacks,
    verbose=1
)

# 评估模型
print("\n 模型评估:")
predictions = model.predict(X_test)

# 天气预测评估

```

```

weather_pred = np.argmax(predictions[0], axis=1)
weather_acc = accuracy_score(y_weather_test, weather_pred)
print(f"天气预测准确率: {weather_acc:.4f}")

# 风力预测评估
wind_pred = np.argmax(predictions[1], axis=1)
wind_acc = accuracy_score(y_wind_test, wind_pred)
print(f"风力预测准确率: {wind_acc:.4f}")

# 降水预测评估
rain_pred = predictions[2].flatten()
rain_mse = mean_squared_error(y_rain_test, rain_pred)
rain_mae = mean_absolute_error(y_rain_test, rain_pred)
print(f"降水预测 MSE: {rain_mse:.4f}")
print(f"降水预测 MAE: {rain_mae:.4f}")

# 详细分类报告
print("\n天气预测详细报告:")
print(classification_report(y_weather_test, weather_pred,
                           target_names=self.weather_encoder.classes_))

self.model = model
self.is_fitted = True
self.use_sequences = use_sequences
self.sequence_length = sequence_length if use_sequences else None

y_test_dict = {
    'weather_output': y_weather_test,
    'wind_output': y_wind_test,
    'rain_output': y_rain_test
}
feature_cols = ['纬度', '经度', 'SO2', 'NO2', 'PM10', '污染程度',
                '时间段', '气温', 'sin_月', 'cos_月', 'sin_日', 'cos_日',
                'sin_星期', 'cos_星期', '污染物总和', '污染物比例_SO2',
                '污染物比例_NO2', '污染物比例_PM10']

return history, (X_test, y_test_dict), feature_cols

def predict(self, input_data):
    """进行预测"""
    if not self.is_fitted:
        raise ValueError("模型尚未训练, 请先调用 train_models 方法")

    # 标准化输入数据
    if self.use_sequences:
        input_resaped = input_data.reshape(-1, input_data.shape[-1])
        input_scaled_resaped = self.scaler_features.transform(input_resaped)
        input_scaled = input_scaled_resaped.reshape(input_data.shape)
    else:
        input_scaled = self.scaler_features.transform(input_data)

    # 预测
    predictions = self.model.predict(input_scaled)

    # 解码预测结果
    weather_pred = self.weather_encoder.inverse_transform(np.argmax(predictions[0], axis=1))
    wind_pred = np.argmax(predictions[1], axis=1)
    rain_pred = predictions[2].flatten()

    return {
        'weather': weather_pred,
        'wind': wind_pred,
        'rain': rain_pred,
        'weather_proba': predictions[0],
        'wind_proba': predictions[1]
    }

def plot_training_history(self, history):
    """绘制训练历史"""
    fig, axes = plt.subplots(2, 2, figsize=(15, 10))

    # 总损失
    axes[0, 0].plot(history.history['loss'], label='Training Loss')

```

```

axes[0, 0].plot(history.history['val_loss'], label='Validation Loss')
axes[0, 0].set_title('Total Loss')
axes[0, 0].legend()

# 天气预测准确率
axes[0, 1].plot(history.history['weather_output_accuracy'], label='Training Accuracy')
axes[0, 1].plot(history.history['val_weather_output_accuracy'], label='Validation Accuracy')
axes[0, 1].set_title('Weather Prediction Accuracy')
axes[0, 1].legend()

# 风力预测准确率
axes[1, 0].plot(history.history['wind_output_accuracy'], label='Training Accuracy')
axes[1, 0].plot(history.history['val_wind_output_accuracy'], label='Validation Accuracy')
axes[1, 0].set_title('Wind Prediction Accuracy')
axes[1, 0].legend()

# 降水预测 MAE
axes[1, 1].plot(history.history['rain_output_mae'], label='Training MAE')
axes[1, 1].plot(history.history['val_rain_output_mae'], label='Validation MAE')
axes[1, 1].set_title('Rain Prediction MAE')
axes[1, 1].legend()

plt.tight_layout()
plt.show()

```

B 模型预测

```

import pandas as pd
import numpy as np
import pickle
from tensorflow.keras.models import load_model
from joblib import load as jb_load
from tensorflow.keras.models import load_model
from datetime import datetime, timedelta

class EnvironmentModelLoader:
    def __init__(self, model_dir='my_environment_model'):
        # 加载 Keras 模型
        self.model = load_model(f"{model_dir}/model.h5")

        # 加载 scaler 和 encoder(关键:使用 joblib.load)
        self.scaler_features = jb_load(f"{model_dir}/scaler_features.pkl")
        self.weather_encoder = jb_load(f"{model_dir}/weather_encoder.pkl")

        # 读取模型元信息
        try:
            with open(f"{model_dir}/meta_info.txt", 'r') as f:
                lines = f.readlines()
                self.use_sequences = False
                self.sequence_length = 12
                for line in lines:
                    if line.startswith("use_sequences="):
                        self.use_sequences = line.strip().split('=')[1].lower() == 'true'
                    elif line.startswith("sequence_length="):
                        self.sequence_length = int(line.strip().split('=')[1])
        except:
            # 默认参数
            self.use_sequences = False
            self.sequence_length = 12

        self.feature_cols = [
            '纬度', '经度', 'SO2', 'NO2', 'PM10', '污染程度',
            '时间段', '气温', 'sin_月', 'cos_月', 'sin_日', 'cos_日',
            'sin_星期', 'cos_星期', '污染物总和',
            '污染物比例_SO2', '污染物比例_NO2', '污染物比例_PM10'
        ]

    def predict_future_days(self, raw_df, days=5):
        """
        预测未来多天的环境数据
        :param raw_df: 历史数据 DataFrame
        :param days: 要预测的天数
        """

```

```

:return: 包含预测结果的 DataFrame 列表
"""
predictions = []
current_df = raw_df.copy()

for day in range(days):
    # 预测下一天的数据
    prediction = self.predict(current_df)

    # 创建新的日期(最后日期+1天)
    last_date = pd.to_datetime(current_df['日期']).iloc[-1]
    new_date = last_date + timedelta(days=1)

    # 预测经纬度变化(基于历史趋势)
    if len(current_df) >= 2:
        # 计算经纬度变化趋势
        lat_trend = (current_df['纬度'].iloc[-1] - current_df['纬度'].iloc[-2]) * 0.8
        lon_trend = (current_df['经度'].iloc[-1] - current_df['经度'].iloc[-2]) * 0.8
        new_lat = current_df['纬度'].iloc[-1] + lat_trend
        new_lon = current_df['经度'].iloc[-1] + lon_trend
    else:
        # 如果没有足够历史数据, 保持原位置或轻微随机变化
        new_lat = current_df['纬度'].iloc[-1] + np.random.normal(0, 0.001)
        new_lon = current_df['经度'].iloc[-1] + np.random.normal(0, 0.001)

    # 创建新的数据行
    new_row = pd.DataFrame({
        '纬度': [new_lat],
        '经度': [new_lon],
        'SO2': [prediction['预测 SO2']],
        'NO2': [prediction['预测 NO2']],
        'PM10': [prediction['预测 PM10']],
        '污染程度': [self.calculate_pollution_level(prediction['预测 SO2'], prediction['预测 NO2'],
prediction['预测 PM10'])],
        '日期': [new_date.strftime('%Y-%m-%d')],
        '时间段': [prediction['预测时间段']],
        '气温': [prediction['预测气温']]
    })

    # 添加到历史数据中(用于下一天的预测)
    current_df = pd.concat([current_df, new_row], ignore_index=True)

    # 保存预测结果
    predictions.append({
        '日期': new_date.strftime('%Y-%m-%d'),
        '预测结果': prediction,
        '预测纬度': new_lat,
        '预测经度': new_lon
    })

return predictions

def calculate_pollution_level(self, so2, no2, pm10):
    """
    根据污染物浓度计算污染程度等级
    """
    # 简单加权计算污染指数
    pollution_index = 0.2*so2 + 0.3*no2 + 0.5*pm10

    if pollution_index < 50:
        return 0 # 优
    elif pollution_index < 100:
        return 1 # 良
    elif pollution_index < 150:
        return 2 # 轻度污染
    elif pollution_index < 200:
        return 3 # 中度污染
    else:
        return 4 # 重度污染

def prepare_input_data(self, raw_df):
    """准备输入数据, 支持时序和非时序两种模式"""

```

```

df = raw_df.copy()
df['日期'] = pd.to_datetime(df['日期'])
df['月'] = df['日期'].dt.month
df['日'] = df['日期'].dt.day
df['星期'] = df['日期'].dt.dayofweek

# 时间特征工程
df['sin_月'] = np.sin(2 * np.pi * df['月'] / 12)
df['cos_月'] = np.cos(2 * np.pi * df['月'] / 12)
df['sin_日'] = np.sin(2 * np.pi * df['日'] / 31)
df['cos_日'] = np.cos(2 * np.pi * df['日'] / 31)
df['sin_星期'] = np.sin(2 * np.pi * df['星期'] / 7)
df['cos_星期'] = np.cos(2 * np.pi * df['星期'] / 7)

# 污染物特征工程
df['污染物总和'] = df['SO2'] + df['NO2'] + df['PM10']
df['污染物比例_SO2'] = df['SO2'] / (df['污染物总和'] + 1e-8)
df['污染物比例_NO2'] = df['NO2'] / (df['污染物总和'] + 1e-8)
df['污染物比例_PM10'] = df['PM10'] / (df['污染物总和'] + 1e-8)

# 提取特征
X = df[self.feature_cols].values

if self.use_sequences:
    # 时序模式:取最后 sequence_length 个数据点
    if len(X) >= self.sequence_length:
        X = X[-self.sequence_length:] # 取最后 12 个时间点
        # 对时序数据进行标准化
        X_resaped = X.reshape(-1, X.shape[-1])
        X_scaled_resaped = self.scaler_features.transform(X_resaped)
        X_scaled = X_scaled_resaped.reshape(X.shape)
        return X_scaled.reshape(1, self.sequence_length, len(self.feature_cols))
    else:
        # 数据不足, 重复最后一行
        last_row = X[-1:]
        X_padded = np.repeat(last_row, self.sequence_length, axis=0)
        X_resaped = X_padded.reshape(-1, X_padded.shape[-1])
        X_scaled_resaped = self.scaler_features.transform(X_resaped)
        X_scaled = X_scaled_resaped.reshape(X_padded.shape)
        return X_scaled.reshape(1, self.sequence_length, len(self.feature_cols))
else:
    # 非时序模式:取最后一个数据点
    X_single = X[-1:] if len(X) > 0 else X[:1]
    X_scaled = self.scaler_features.transform(X_single)
    return X_scaled

def predict_from_history(self, raw_df):
    """基于历史数据预测未来的环境参数"""
    # 获取最后一条记录作为基准
    last_row = raw_df.iloc[-1] if len(raw_df) > 0 else raw_df.iloc[0]

    # 基于历史趋势预测污染物浓度
    if len(raw_df) >= 2:
        prev_row = raw_df.iloc[-2]
        # 计算变化趋势
        so2_trend = (last_row['SO2'] - prev_row['SO2']) * 0.7
        no2_trend = (last_row['NO2'] - prev_row['NO2']) * 0.7
        pm10_trend = (last_row['PM10'] - prev_row['PM10']) * 0.7
        temp_trend = (last_row['气温'] - prev_row['气温']) * 0.5
    else:
        # 如果只有一条记录, 假设轻微变化
        so2_trend = np.random.normal(0, 5)
        no2_trend = np.random.normal(0, 5)
        pm10_trend = np.random.normal(0, 10)
        temp_trend = np.random.normal(0, 2)

    # 预测污染物浓度(确保非负)
    predicted_so2 = max(0, last_row['SO2'] + so2_trend)
    predicted_no2 = max(0, last_row['NO2'] + no2_trend)
    predicted_pm10 = max(0, last_row['PM10'] + pm10_trend)

```



```

# 预测气温
predicted_temp = last_row['气温'] + temp_trend

# 预测时间段切换(0->1 或 1->0)
predicted_time_period = 1 - last_row['时间段']

return {
    'predicted_so2': predicted_so2,
    'predicted_no2': predicted_no2,
    'predicted_pm10': predicted_pm10,
    'predicted_temp': predicted_temp,
    'predicted_time_period': predicted_time_period
}

def predict(self, raw_df):
    """
    完整预测方法
    预测: SO2, NO2, PM10, 时间段, 气温, 风向编码, 风力编码, 降水强度, 天气
    """
    # Step 1: 准备输入数据
    X_input = self.prepare_input_data(raw_df)

    # Step 2: 模型预测(天气、风力、降水)
    preds = self.model.predict(X_input, verbose=0)

    # 模型输出解析:
    # preds[0]: 天气预测概率 (weather_output)
    # preds[1]: 风力预测概率 (wind_output)
    # preds[2]: 降水强度预测 (rain_output)

    weather_pred = self.weather_encoder.inverse_transform(np.argmax(preds[0], axis=1))[0]
    wind_force_pred = int(np.argmax(preds[1], axis=1)[0])
    rain_intensity_pred = float(preds[2].flatten()[0])

    # Step 3: 基于历史数据预测其他参数
    historical_predictions = self.predict_from_history(raw_df)

    # Step 4: 基于天气状况调整预测
    weather_impact = self.apply_weather_impact(weather_pred, historical_predictions)

    # Step 5: 预测风向编码(基于风力和地理位置)
    last_row = raw_df.iloc[-1]
    wind_direction_pred = self.predict_wind_direction(
        wind_force_pred,
        last_row['纬度'],
        last_row['经度'],
        weather_pred
    )

    # Step 6: 返回完整预测结果
    return {
        '预测 SO2': float(weather_impact['so2']),
        '预测 NO2': float(weather_impact['no2']),
        '预测 PM10': float(weather_impact['pm10']),
        '预测时间段': int(historical_predictions['predicted_time_period']), # 0=白天, 1=晚上
        '预测气温': float(weather_impact['temperature']),
        '预测风向编码': wind_direction_pred,
        '预测风力编码': wind_force_pred,
        '预测降水强度': rain_intensity_pred,
        '预测天气': weather_pred,
        '预测经度': last_row['经度'],
        '预测纬度': last_row['纬度'], # 使用最后一条记录的经纬度
        '预测日期': pd.to_datetime(last_row['日期']) + timedelta(days=1),
        '预测污染程度': self.calculate_pollution_level(
            weather_impact['so2'],
            weather_impact['no2'],
            weather_impact['pm10']
        ),

        # 额外信息
    }

```

```

        '天气预测置信度': float(np.max(preds[0])),
        '风力预测置信度': float(np.max(preds[1]))
    }

def apply_weather_impact(self, weather, historical_pred):
    """根据天气状况调整污染物和气温预测"""
    so2 = historical_pred['predicted_so2']
    no2 = historical_pred['predicted_no2']
    pm10 = historical_pred['predicted_pm10']
    temp = historical_pred['predicted_temp']

    # 天气对污染物的影响
    if weather in ['雨', '雪']:
        # 降水会冲刷污染物
        pollution_factor = 0.6
        temp_adjustment = -3 # 降水降温
    elif weather == '雾霾沙尘':
        # 雾霾会增加污染物浓度
        pollution_factor = 1.5
        temp_adjustment = -1
    elif weather == '晴':
        # 晴天有利于污染物扩散但也可能积累
        pollution_factor = 1.1
        temp_adjustment = 2 # 晴天升温
    elif weather == '多云':
        pollution_factor = 0.9
        temp_adjustment = 0
    elif weather == '阴':
        pollution_factor = 1.2 # 阴天不利于扩散
        temp_adjustment = -1
    else:
        pollution_factor = 1.0
        temp_adjustment = 0

    return {
        'so2': max(0, so2 * pollution_factor),
        'no2': max(0, no2 * pollution_factor),
        'pm10': max(0, pm10 * pollution_factor),
        'temperature': temp + temp_adjustment
    }

def predict_wind_direction(self, wind_force, latitude, longitude, weather):
    """预测风向编码"""
    # 基于地理位置的基础风向倾向
    if latitude > 35: # 北方地区
        base_directions = [0, 1, 7] # 偏北风
    else: # 南方地区
        base_directions = [3, 4, 5] # 偏南风

    # 风力影响
    if wind_force <= 2:
        # 微风时方向较随机
        direction_pool = list(range(8))
    elif wind_force <= 5:
        # 中等风力时有一定规律
        direction_pool = base_directions + [d for d in range(8) if d not in base_directions][:2]
    else:
        # 强风时方向相对固定
        direction_pool = base_directions

    # 天气影响
    if weather in ['雨', '雪']:
        # 降水天气通常伴随特定风向
        if latitude > 35:
            direction_pool = [0, 1, 2] # 北方降水多为北风
        else:
            direction_pool = [4, 5, 6] # 南方降水多为南风

    return int(np.random.choice(direction_pool))

def batch_predict(self, data_list):
    """批量预测"""
    results = []

```

```

for data in data_list:
    try:
        result = self.predict(data)
        results.append(result)
    except Exception as e:
        print(f"预测失败: {e}")
        results.append(None)
return results

if __name__ == "__main__":
    # 创建预测器
    loader = EnvironmentModelLoader(model_dir='my_environment_model')
    raw_input = pd.read_csv('广运潭_latest_12.csv') # 假设输入数据存储在 CSV 文件中

    future_predictions = loader.predict_future_days(raw_input, days=5)

    print("✔ 未来 5 天环境预测结果:")
    for i, pred in enumerate(future_predictions, 1):
        result = pred['预测结果']
        print(f"\n 第{i}天预测 ({pred['日期']}):")
        print("-" * 40)

        # 位置信息
        print(f" 位置预测:")
        print(f"    纬度: {pred['预测纬度']:.6f}")
        print(f"    经度: {pred['预测经度']:.6f}")

        # 污染物预测
        print(f"\n 污染物浓度预测:")
        print(f"    SO2: {result['预测 SO2']:.1f} μg/m³")
        print(f"    NO2: {result['预测 NO2']:.1f} μg/m³")
        print(f"    PM10: {result['预测 PM10']:.1f} μg/m³")

        # 气象预测
        print(f"\n 气象条件预测:")
        time_desc = "白天" if result['预测时间段'] == 0 else "晚上"
        print(f"    时间段: {result['预测时间段']} ({time_desc})")
        print(f"    气温: {result['预测气温']:.1f} °C")
        print(f"    天气: {result['预测天气']}")

        # 风力条件
        print(f"\n 风力条件预测:")
        print(f"    风向编码: {result['预测风向编码']}")
        print(f"    风力编码: {result['预测风力编码']}")
        print(f"    降水强度: {result['预测降水强度']:.2f} mm/h")

        # 预测置信度
        print(f"\n 预测置信度:")
        print(f"    天气预测置信度: {result['天气预测置信度']:.3f}")
        print(f"    风力预测置信度: {result['风力预测置信度']:.3f}")

    except Exception as e:
        print(f"✗ 预测失败: {e}")
        import traceback
        traceback.print_exc()

    print("\n" + "="*60)
    print(" 预测说明:")
    print("• 时间段: 0=白天, 1=晚上")
    print("• 污染物浓度单位: μg/m³")
    print("• 气温单位: °C")
    print("• 降水强度单位: mm/h")
    print("• 风向编码: 0-7 (0=北, 1=东北, 2=东, 3=东南, 4=南, 5=西南, 6=西, 7=西北)")
    print("• 风力编码: 数值越大风力越强")
    print("• 预测基于深度学习模型和历史数据趋势分析")
    # 保存 txt

```